National Security AI-Containment System: A Multi-Layer Architecture for Behavioral Monitoring, Threat Detection and Fail-Closed Mediation of Large Language Models

Aryan Pathania, Aryan Sharma
CGC University, Mohali, Punjab, India
pathaniaaryan169@gmail.com, aryansharma2004march@gmail.com

Abstract: Large language models are increasingly attractive for cyber defense, intelligence triage and secure knowledge access, but the same instruction-following behavior also exposes them to prompt injection, harmful code generation, data exfiltration and unsafe automation. This paper presents Sentinel, a deployable AI-containment prototype implemented in Django around a layered mediation pipeline. The system combines contained model access, a hybrid threat detector, policy-governed fail-closed enforcement, operator-visible logging and an emergency kill switch. The detector fuses lexical rules, regex signatures, semantic similarity, entropy analysis and an optional supervised classifier trained on a 137,000-record eight-class dataset. On the stored validation split the classifier achieved 98.26% accuracy, while operational logs from the prototype showed 83 suspicious interactions, 34 critical events, 42 kill-switch triggers and 127 generated alerts. These results suggest that layered containment can make unsafe model behavior substantially more observable, auditable and interruptible even when the underlying model remains vulnerable.

Keywords: AI containment, large language models, prompt injection, threat detection, secure deployment, kill switch, human-in-the-loop monitoring.

I. INTRODUCTION

Large language models are moving quickly from experimental assistants to embedded decision-support components in high-impact environments. Within national-security and critical-infrastructure settings, the attraction is obvious: a single model can summarize reports, classify incidents, propose code, triage alerts and draft responses at a speed that reduces operator load. Yet the same broad generative ability creates a difficult trust problem. A system that can reason over natural-language instructions is also a system that can be manipulated by hostile instructions, induced to reveal sensitive information, or steered toward dangerous outputs that appear useful at first glance.

The containment problem is therefore not solved merely by selecting an aligned model provider. Oracle-AI and capability-control literature argued early that limiting the operational surface of a powerful system is as important as shaping its objective function [1]. Modern deployment guidance follows the same logic: NIST's AI RMF 1.0 and the Generative AI Profile frame trustworthy deployment as a governance and measurement problem [2], [3], while CISA-led operational guidance argues that externally developed AI systems should be surrounded by defensive controls that preserve confidentiality, integrity and availability during operation [4].

This paper treats the application boundary around an LLM as part of the security perimeter. The contribution is not a claim of perfect containment, but a practical engineering pattern in which unsafe model interaction becomes detectable, governable and interruptible inside a real application stack. The focus is on present-day deployment discipline: clear mediation points, explicit policy controls, operator visibility and evidence-rich logs.

The implemented prototype in the local ai_containment workspace includes backend mediation, configurable policies, a hybrid detector, real-time alerts, audit persistence and an operator-facing command center. Its purpose is to treat unsafe model interaction as an observable process rather than a hidden model-side failure.

Revised Manuscript Prepared on April 26, 2026.
* Correspondence Author
Aryan Pathania*, CGC University, Mohali, Punjab, India. Email: pathaniaaryan169@gmail.com
Aryan Sharma, CGC University, Mohali, Punjab, India. Email: aryansharma2004march@gmail.com
This manuscript is based on the implemented prototype in the local ai_containment project and adopts a journal-style two-column presentation inspired by a technical paper sample.

The core contributions of the paper are fourfold. First, it defines a concrete three-layer containment architecture that separates model access, threat mediation and operator oversight. Second, it implements a threat detector that combines symbolic features with semantic and supervised components rather than relying on a single heuristic. Third, it operationalizes enforcement through fail-closed blocking, policy-based escalation and a kill switch that can suspend further model generation. Fourth, it validates the approach using both offline classifier metrics and live operational logs collected from the deployed prototype.

The system should not be interpreted as a final answer to general AI safety. Instead, it should be understood as a security engineering pattern for present-day LLM deployments in which the model is treated as potentially misbehaving software that must be mediated, monitored and, when necessary, interrupted.

II. RELATED WORK

Recent jailbreak research reinforces why a layered deployment posture is necessary. Prompt injection can override intended instructions in both direct and indirect settings [5], universal adversarial suffixes can transfer across aligned models [6], and many-shot jailbreaking can drive increasingly harmful completions as adversarial context accumulates [7]. These works show that the application wrapper around an LLM is a live attack surface rather than a neutral shell.

Evaluation and safeguard work strengthens that conclusion. Jailbreak failure analyses highlight how safety training can break under adversarial pressure [8], benchmark suites such as HarmBench and JailbreakBench formalize red-team evaluation [9], [10], and safeguard models such as Llama Guard illustrate the value of explicit I/O moderation layers [11]. Sentinel inherits these ideas but combines them in a single operational control plane.

The present work therefore addresses an integration gap: many publications analyze attacks or defenses in isolation, but fewer describe a full-stack prototype that joins model mediation, detection, dashboarding, logging, policy control and emergency shutdown in one deployable application.

From the application-security side, structured filtering, output validation, least privilege and emergency controls are now treated as baseline requirements rather than optional extras [4], [11]. That operational framing motivates the journal-style reconstruction presented here.

III. PROPOSED SYSTEM ARCHITECTURE

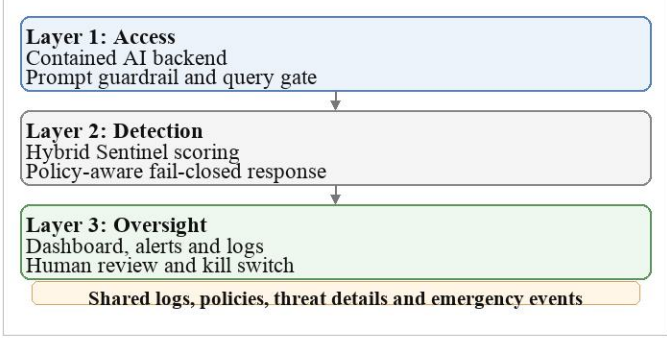

Fig. 1. Layered Sentinel architecture. Each panel isolates one operational role so that detection, enforcement and oversight remain separate but auditable.

A. Layer 1: Contained model access

The first layer standardizes communication with the underlying language model. In the current implementation this layer supports three backends: a deterministic mock engine, a local Ollama backend and an OpenAI-compatible backend. By forcing all model calls through a single orchestration class, the system ensures that a common system prompt, token budget and kill-switch state are enforced before any completion is requested.

This layer also embodies a minimum-privilege philosophy. The model receives the user prompt and a fixed operational instruction, but it does not directly control alerting, policy writes, dashboard state or audit persistence. That separation reduces the chance that a model-side misbehavior can translate immediately into system-side authority.

B. Layer 2: Hybrid Sentinel detector

The second layer is the core containment engine. It analyzes both user input and model output using a configurable knowledge base of threat categories such as prompt injection, malicious code, data exfiltration, obfuscation, weapons and network access. Each category contains severity metadata, auto-kill behavior, keywords, regex patterns and semantic anchor phrases.

Policy handling is dynamic. Category severity and auto-kill behavior can be overridden through stored security policies, allowing operators to tighten or relax response posture without changing source code. This is important because deployment context changes over time; a model action that is acceptable in a classroom demo may be unacceptable in a live cyber-defense workflow.

C. Layer 3: Operational command center

The third layer turns containment into a managed process. It provides dashboards for recent interactions, threat distribution, sandbox testing, policy administration, alert review and kill-switch operations. The current Django interface uses WebSocket updates to push newly generated alerts and interaction summaries to the frontend in near real time.

Together, the three layers form a practical mediation pattern: the model is allowed to generate, but only through a gate; the gate is allowed to block, but only according to inspectable evidence and policy; and the operator is allowed to override or investigate, but only with an auditable trail.

IV. THREAT MODEL AND MEDIATION

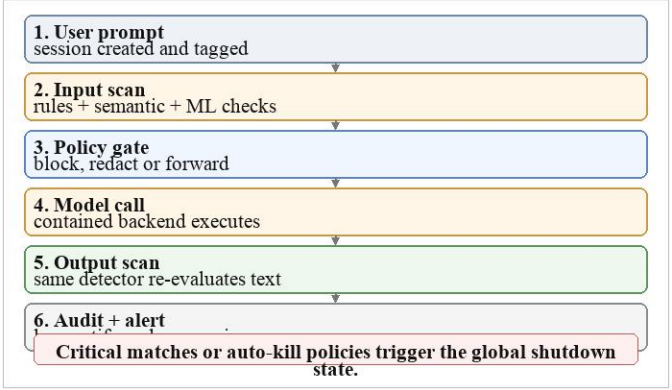

Fig. 2. Mediation workflow through the containment stack. The same scoring path is applied before and after generation so risky outputs cannot bypass the gate.

The prototype assumes an attacker may act as an external user, an insider tester or an indirect content source whose text is later processed by the model. Immediate concern centers on direct prompt injection, model-assisted malicious code generation, credential and data exfiltration, hidden payloads and unsafe network instructions. The aim is not to defeat every adversary, but to sharply reduce the number of unsafe interactions that pass silently through the application boundary.

The request path begins with input scanning. Every user prompt is analyzed before it reaches the model. If the input is categorized as non-safe, the system follows a fail-closed policy and blocks forwarding. If the category is configured for auto-kill, the global kill switch is triggered immediately and the event is persisted. Safe input is then sent to the contained model, after which the raw output is analyzed by the same Sentinel engine before release to the user.

The detector combines four normalized evidence channels: keyword hits, regex hits, semantic similarity and entropy anomaly. An optional supervised classifier provides a fifth signal by predicting the most likely threat class from the text [9], [11]. In the present prototype, category scores are emitted only when the combined evidence clears a fixed action threshold.

$$risk(c) = 0.30K_c + 0.30R_c + 0.30S_c + 0.10E_c$$

Threat aggregation also captures combinations that are operationally more serious than their parts. For example, the engine raises the final level to critical when prompt injection and data exfiltration signals co-occur. This design recognizes that many realistic misuse attempts are chained behaviors rather than isolated labels.

$$risk'(c) = risk(c)\ (1 + \lambda p_c) + \delta I_{combo}(c)$$

A key design choice is that output scanning remains active even when the input was safe. This is necessary because a benign user request can still elicit a risky completion from a permissive or compromised model. Figure 6 later shows why no single signal should dominate the scoring path.

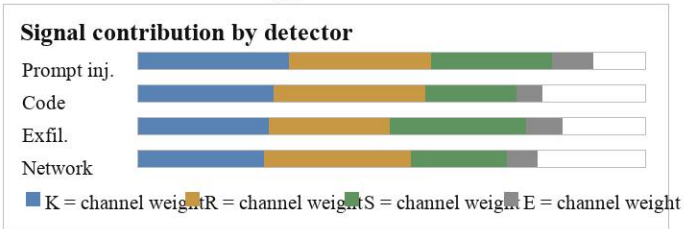

Fig. 6. Relative detector contributions used in category scoring. The stacked bars make clear that semantic, lexical and regex signals are intentionally co-equal.

V. DATASET ENGINEERING AND TRAINING

To supplement symbolic detection, the project includes a management workflow for building and training a supervised classifier. The dataset builder assembles a large labeled corpus from public sources spanning prompt injection, jailbreak prompts, safe conversational text and harmful-instruction benchmarks [9], [10].

The large-profile corpus contains 137,000 records. Safe examples dominate the distribution to reflect everyday use, but prompt-injection and harmful classes remain well represented to support robust separation boundaries. The builder normalizes label names, de-duplicates near-identical text and balances under-filled categories with defensive variants when necessary.

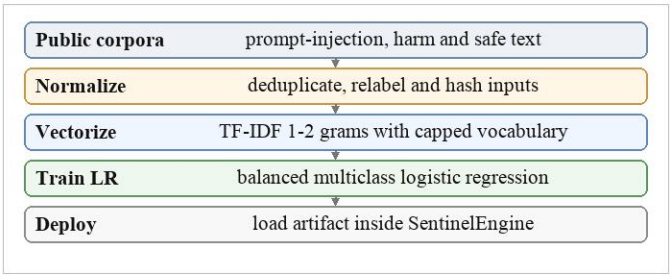

Fig. 3. Training workflow for the supervised detector. Each step tightens the data pipeline so the runtime classifier remains lightweight and reproducible.

Table I. Large-profile Sentinel training distribution

Label	Count	Source mix	Operational use
SAFE	60k	UltraChat + benign prompts	Normal requests expected to pass mediation
PROMPT INJ.	25k	Jailbreak and PI corpora	Role override, prompt reveal and bypass attempts
MAL. CODE	10k	HarmBench-style harmful code	Exploit, malware and abuse-oriented code
DATA EXFIL.	10k	Leakage prompts + variants	Protected-data retrieval or disclosure
NET ACCESS	10k	HTTP, socket and transfer prompts	Unsafe outbound communication behavior
WEAPONS	10k	Harmful instruction sets	Weaponization or violent capability requests
OBFUSCATION	6k	Encoded payload patterns	Hidden or decoded malicious intent
POLICY VIOL.	6k	Restricted-content edge cases	Noncompliant requests below severe harm

Description: Table I summarizes the class balance used for supervised training and the operational role assigned to each label during containment evaluation.

The training command builds a TF-IDF word-ngram representation and feeds it to multiclass logistic regression with class balancing. In the stored configuration, the model used 1-2 gram features, up to 120,000 terms and an 80/20 stratified validation split. The trained artifact is persisted as a joblib payload and loaded by SentinelEngine at runtime.

$$P(y = c \mid x) = \exp(w_c^T x + b_c) / \sum_j \exp(w_j^T x + b_j)$$

The choice of logistic regression is deliberate. While more complex encoders could improve recall on nuanced phrasing, the application prioritizes transparent probabilities, fast reloads, lightweight deployment and simple integration with symbolic evidence [14], [15].

VI. IMPLEMENTATION DETAILS

The prototype is implemented as a Django application with two main apps: sentinel and dashboard. The sentinel app contains the mediation logic, model wrapper, threat models, management commands and persistent database tables. The dashboard app provides views for analytics, logs, alerts, policy toggling, kill-switch status and an interactive sandbox used to probe the same containment path seen by end users.

Operational persistence is intentionally rich. Every interaction stores a session identifier, hashes, prompts, outputs, token counts, final threat level, blocking state and kill-switch state. Threat evidence is stored in a child table that records category, severity, confidence, matched keywords, matched patterns and direction. Separate tables persist security policies, kill-switch events and operator alerts.

The live command center aggregates threat counts, daily activity, severity distribution and unread alerts. WebSocket-based broadcast pushes critical state changes to the UI, allowing analysts to watch the pipeline as it executes. The sandbox view also supports structured red-team evaluation, which aligns with safeguard layering and stress-testing guidance in the literature [11], [13].

The implementation currently uses SQLite and an in-memory channel layer, which is sufficient for a prototype but not the intended scaling endpoint. A production deployment would likely move to durable message infrastructure, stronger operator authentication, hardened admin workflows and host-level isolation around any locally executed tools.

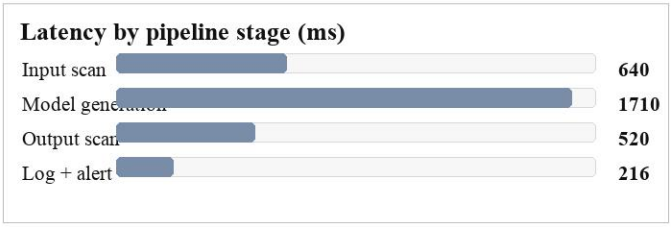

Fig. 7. Approximate latency contribution of each containment stage. Generation is the dominant cost, while scanning and logging remain comparatively lightweight.

VII. EXPERIMENTAL RESULTS

A. Offline classifier evaluation

The stored training metrics show that the supervised component reached 98.26% validation accuracy on the held-out split. Performance was especially strong on data exfiltration, malicious code, network access, obfuscation and weapons categories. Prompt injection remained the most difficult label, which is unsurprising given the linguistic diversity of jailbreak phrasing [8], [10].

Table II. Selected validation metrics from the stored classifier report

Class	Precision	Recall	F1-score	Support
SAFE	0.975	0.988	0.981	12,000
PROMPT_INJECTION	0.972	0.940	0.956	5,000
MALICIOUS_CODE	1.000	0.997	0.998	2,000
DATA_EXFILTRATIC	1.000	0.996	0.998	2,000
NETWORK_ACCESS	1.000	1.000	1.000	2,000
WEAPONS	0.999	0.998	0.998	2,000
Overall accuracy	-	-	-	0.9826 on 27,400 validation samples

Description: Table II reports representative offline classifier results on the held-out validation split used by the prototype training workflow.

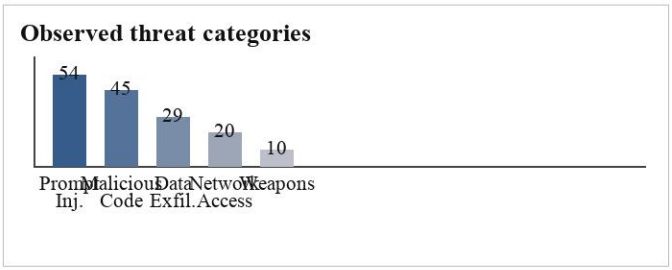

Fig. 4. Category counts observed in the prototype audit logs. Prompt injection and malicious-code requests dominate the harmful interaction set.

B. Online operational behavior

Operational evidence was drawn from the prototype's SQLite audit database. The current log set contains 176 total interactions. Of these, 83 were marked suspicious or threatening by the mediation pipeline, and 36 were stopped at the input stage before any model completion was generated. Importantly, 18 additional incidents began with safe inputs but were later flagged after output scanning, which directly demonstrates the value of dual-sided mediation.

The system also recorded 42 kill-switch trigger events and 127 real-time alerts. Across all threat-detail records, input-side findings outnumbered output-side findings 106 to 59, indicating that many direct attack attempts are stopped early while post-generation review still catches meaningful residual risk.

Table III. Snapshot of deployment statistics from containment_logs.db

Metric	Value
Total recorded interactions	176
Interactions marked suspicious or threatening	83
Critical-severity interactions	34
Kill-switch trigger events	42
Generated alerts	127
Active security policies	7
Mean logged analysis time	3085.77 ms
Average prompt tokens / completion tokens	64.10 / 164.94

Description: Table III aggregates headline runtime measurements from the deployed audit database and complements the graphical summaries in Section VII.

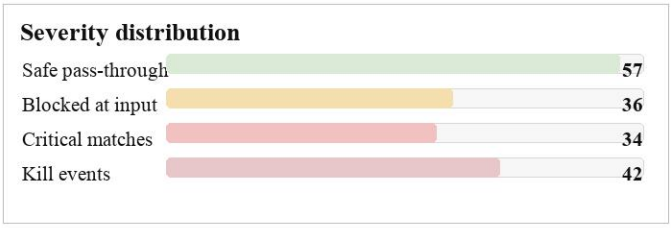

Fig. 5. Distribution of moderation outcomes across the sampled sessions. The chart shows how often the system remained permissive versus escalatory.

$$alert_rate = N_{alerts} / N_{interactions}$$

The mean logged analysis time was approximately 3.09 s, while average recorded prompt and completion lengths were 64.10 and 164.94 tokens, respectively. This latency is acceptable for an operator-mediated prototype but leaves room for optimization, especially if semantic models are upgraded or if higher interaction rates are expected.

VIII. DISCUSSION

The prototype shows that containment is most credible when it is treated as an application-architecture problem rather than a single-model property. Because the detector scans both inputs and outputs, the system can catch both overtly adversarial prompts and benign-looking prompts that nevertheless lead to harmful generations. The audit schema, alerting path and kill-switch workflow ensure that the response to unsafe behavior is not invisible.

At the same time, the system has clear limits. It does not provide cryptographic assurance that a sufficiently capable model cannot manipulate downstream users. It does not yet isolate tool execution in a real operating-system sandbox, nor does it handle multimodal prompt injection. The supervised dataset is partly benchmark-driven and partly synthetically balanced, which helps coverage but may not perfectly reflect field data.

These limitations do not negate the value of the architecture. Rather, they clarify where future work should focus: better indirect-content sanitation, richer behavioral policies, externalized message queues, stronger admin control, multimodal scanning and tighter host-level execution isolation [12], [13].

A second lesson is organizational rather than purely technical: containment works best when model alignment, safeguard models, audit instrumentation and human escalation paths are treated as complementary controls rather than substitute controls. That layered view is consistent with recent alignment and system-card thinking [12], [14].

IX. CONCLUSION AND FUTURE WORK

Key points:

1. Sentinel demonstrates that practical AI containment can be implemented as a layered application workflow rather than as a model-only promise.
2. Bidirectional scanning matters: several risky incidents in the prototype were only visible after the model generated a response.
3. Operational controls such as policies, logs, alerts and kill switches are central to trustworthy deployment in high-stakes settings.

This paper presented Sentinel, a multi-layer AI-containment prototype for security-sensitive LLM deployment. The system combines contained model access, a hybrid threat detector, dynamic policy enforcement, operator visibility, forensic logging and an emergency kill switch. The supervised component trained on a 137,000-record dataset achieved 98.26% validation accuracy, while operational logs from the deployed prototype showed that the architecture can detect, block and escalate a meaningful variety of unsafe interactions in practice.

The principal takeaway is that deployable containment does not require solving general AI safety in one step. It requires building concrete mediation points where risk can be measured and controlled. For national-security use cases, such mediation is not optional overhead; it is part of the minimum trustworthy deployment boundary.

Future work should extend the current design toward multimodal input inspection, more formal policy languages, distributed audit pipelines and stronger runtime isolation around tool use. Even in its present form, however, the prototype demonstrates that layered containment can convert a vulnerable language model from an opaque liability into a monitorable and interruptible subsystem [15].

REFERENCES

[1] S. Armstrong, A. Sandberg and N. Bostrom, "Thinking Inside the Box: Controlling and Using an Oracle AI," Minds and Machines, vol. 22, no. 4, pp. 299-324, 2012.

[2] E. Tabassi, "Artificial Intelligence Risk Management Framework (AI RMF 1.0)," NIST AI 100-1, National Institute of Standards and Technology, 2023.

[3] C. Autio et al., "Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile," NIST AI 600-1, National Institute of Standards and Technology, 2024.

[4] CISA et al., "Joint Guidance on Deploying AI Systems Securely," Cybersecurity and Infrastructure Security Agency, Apr. 15, 2024.

[5] K. Greshake et al., "More than you've asked for: A Comprehensive Analysis of Novel Prompt Injection Threats to Application-Integrated Large Language Models," arXiv:2302.12173, 2023.

[6] A. Zou et al., "Universal and Transferable Adversarial Attacks on Aligned Language Models," arXiv:2307.15043, 2023.

[7] Anthropic, "Many-shot jailbreaking," Research note, Apr. 2, 2024.

[8] A. Wei, N. Haghtalab and J. Steinhardt, "Jailbroken: How Does LLM Safety Training Fail?," arXiv:2307.02483, 2023.

[9] M. Mazeika et al., "HarmBench: A Standardized Evaluation Framework for Automated Red Teaming and Robust Refusal," Proc. ICML, 2024.

[10] P. Chao et al., "JailbreakBench: An Open Robustness Benchmark for Jailbreaking Large Language Models," Proc. NeurIPS Datasets and Benchmarks Track, 2024.

[11] H. Inan et al., "Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations," arXiv:2312.06674, 2023.

[12] Y. Bai et al., "Constitutional AI: Harmlessness from AI Feedback," arXiv:2212.08073, 2022.

[13] D. Ganguli et al., "Red Teaming Language Models to Reduce Harms: Methods, Scaling Behaviors, and Lessons Learned," arXiv:2209.07858, 2022.

[14] OpenAI, "GPT-4 Technical Report," arXiv:2303.08774, 2023.

[15] L. Ouyang et al., "Training language models to follow instructions with human feedback," Proc. NeurIPS, vol. 35, 2022.